

Análise de Segurança do Sistema

Início da análise 04/10/2024

Finalizado 07/10/2024

Nome do sistema; agendamento

Tecnologia utilizada: php

Durante a auditoria de segurança do sistema, foram identificadas vulnerabilidades críticas relacionadas à manipulação de dados e segurança de transações. A seguir, uma análise detalhada dos problemas encontrados e as respectivas recomendações para mitigá-los.

1. teste SQL Injection

Foi observado que o código, em especial nas funcionalidades de login e agendamento, faz uso de requisições SQL sem o devido tratamento por meio de **prepared statements** ou de um ORM (Object-Relational Mapping). A ausência desses mecanismos pode expor o sistema a ataques de SQL Injection, onde entradas maliciosas podem ser injetadas diretamente nas consultas SQL, comprometendo a integridade do banco de dados. Um exemplo seria o arquivo `Login.php`, onde presumimos que as variáveis `email` e `senha` são inseridas diretamente nas consultas, sem sanitização. Para mitigar essa vulnerabilidade, é fundamental implementar **prepared statements** em todas as consultas SQL, assegurando que os dados inseridos pelos usuários sejam devidamente tratados antes de interagir com o banco de dados.

2. teste Cross-Site Scripting (XSS)

A análise revelou que não há mecanismos de sanitização de dados de entrada e saída em alguns arquivos, como o `Login.html` e o `main.js`. Essa ausência de tratamento pode permitir que scripts maliciosos sejam injetados e executados nas páginas, comprometendo a segurança da aplicação e dos usuários. No arquivo `main.js`, por exemplo, a função de exibição de horários no calendário (`dayTable.innerHTML = i;`) insere diretamente valores na página sem sanitização, o que pode ser explorado por scripts maliciosos. A recomendação é utilizar funções de sanitização, como `encodeURIComponent`, ou bibliotecas específicas para limpeza de dados no HTML, garantindo que qualquer dado injetado no DOM esteja livre de códigos maliciosos.

3. Teste de Cross-Site Request Forgery (CSRF)

Foi identificada a falta de proteção contra ataques **Cross-Site Request Forgery (CSRF)**, especialmente em requisições POST, como nos formulários de login (`Login.html`) e de agendamento (`createAgendamento.php`). Sem a inclusão de tokens de verificação, o sistema fica vulnerável a ataques que exploram a confiança do servidor em solicitações autenticadas indevidamente. A solução para este problema é a implementação de **tokens CSRF** em todos os formulários sensíveis, garantindo que as requisições originadas sejam legítimas e provenientes do usuário autorizado, minimizando o risco de ataques CSRF.

4. Teste de Validação de Entrada

A validação de entrada mostrou-se inadequada tanto no lado cliente quanto no servidor, especialmente em campos de e-mail e senha no `Login.html` e nos dados enviados via POST para a criação de agendamentos. A falta de mecanismos de validação adequados pode permitir que entradas incorretas ou maliciosas sejam processadas pelo sistema, causando falhas ou exposições. Para evitar isso, recomenda-se a implementação de validações robustas no lado cliente, utilizando expressões regulares, e no lado servidor, com funções como `filter_var()` no PHP, para garantir que os dados sejam corretamente validados e sanitizados.

5. Teste de Gerenciamento de Sessões

Não foram identificadas práticas adequadas de gerenciamento de sessões, como a configuração de cookies seguros ou o uso de sessões expiráveis. A ausência dessas medidas pode expor o sistema a vulnerabilidades como o **session fixation**, onde um invasor poderia se apropriar de uma sessão ativa do usuário. Recomenda-se marcar os cookies de sessão com as flags **HttpOnly** e **Secure**, especialmente quando o sistema estiver rodando sob HTTPS. Além disso, é importante configurar o tempo de expiração das sessões de forma adequada e regenerar a sessão após o login, evitando que sessões antigas sejam reutilizadas indevidamente.

Conclusão e Recomendações Finais

Para fortalecer a segurança do sistema, as seguintes ações são recomendadas:

- Implementar **prepared statements** em todas as consultas SQL para prevenir SQL Injection.

- Sanitizar todas as entradas e saídas de dados que interagem com o DOM para evitar ataques XSS.
- Adicionar **tokens CSRF** em formulários e verificar sua validade no backend para garantir a segurança contra ataques de Cross-Site Request Forgery.
- Implementar validações de entrada tanto no frontend quanto no backend para assegurar que os dados fornecidos pelos usuários estejam corretos e seguros.
- Assegurar que o gerenciamento de sessões seja feito de forma segura, com cookies marcados corretamente e sessões expiráveis.

Ferramentas usadas:

SQLMap

Acesso: <https://sqlmap.org>

Burp Suite

Acesso: <https://portswigger.net/burp>

OWASP ZAP

Acesso: <https://owasp.org/www-project-zap/>

XSSer

<https://github.com/epsylon/xsser>

CSRFTester (OWASP)

Acesso: <https://owasp.org/www-project-csrftester/>

SSL Labs Test

Acesso: <https://www.ssllabs.com/ssltest/>

jSQL Injection

Acesso: <https://github.com/ron190/jsql-injection>

ESAPI (OWASP Enterprise Security API)

Acesso: <https://owasp.org/www-project-enterprise-security-api/>

Pesquisar cada uma das ferramentas para saber quais foram os tipos de testes e como melhorar o código